

LABORATÓRIO DE ENGENHARIA DE SOFTWARE

Nexus Estates

Guia de Execução, Arquitetura e Testes

Gestão centralizada de reservas e ocupação para imóveis. Um ecossistema de microserviços focado em consistência de dados, prevenção de erros de reserva e integração simulada com plataformas externas.

Repositório GitHub:

<https://github.com/kanekitakitos/Nexus-Estates>

Docentes

José Barateiro
Marco Giunti

Estudantes

Brandon Mejia – 79261
Luís Moreira – 81432
Miguel Correia – 71369
Tiago Antunes – 76920

Contents

1	Stack tecnológica	2
1.1	Backend	2
1.2	Frontend	2
1.3	Infraestrutura	2
1.4	Justificação das Escolhas	2
2	Equipa, Gestão e Integração Contínua	3
2.1	Papéis da Equipa	3
2.2	Gestão de Projeto (SCRUM e Requisitos)	3
2.3	Integração Contínua (CI)	3
3	Objetivo	4
4	Como correr o projeto (Docker – Deploy/Demonstração)	4
4.1	Comando único (levanta tudo)	4
4.2	Acessos e Conta de Teste	4
4.3	Parar e limpar	4
4.4	Como a comunicação funciona (rede interna)	4
4.5	Configuração por variáveis de ambiente (.env por serviço)	4
5	Como correr em DEV (infra em Docker + apps localmente)	5
5.1	Levantar apenas Postgres + RabbitMQ	5
5.2	Backend local (Spring Boot)	5
5.3	Frontend local (Next.js + Bun)	5
6	Padrões de projeto e práticas (backend)	5
6.1	Separação por camadas (MVC + Service + Repository)	5
6.2	Proxy Pattern (HTTP declarativo entre microserviços)	5
6.3	Strategy Pattern (ex.: faturação no finance-service)	5
6.4	Resiliência (Circuit Breaker + Retry)	6
6.5	Eventos assíncronos e consistência eventual	6
7	Testes (backend)	7
7.1	api-gateway	7
7.2	booking-service	7
7.3	property-service	7
7.4	sync-service	7
7.5	finance-service	7
7.6	user-service	7
7.7	Como correr os testes (backend)	7
8	Testes (frontend) — integração/E2E (Playwright)	8
8.1	Como correr testes do frontend	8
	Referências	9

1 Stack tecnológica

1.1 Backend

- Java 23, Spring Boot 3.3.x
- Spring Cloud Gateway (API Gateway)
- Spring Data JPA + PostgreSQL 16 (persistência)
- Flyway (migrações)
- RabbitMQ (AMQP) para eventos assíncronos
- SpringDoc OpenAPI/Swagger UI
- Resilience4j (Circuit Breaker/Retry) no **sync-service** para integrações externas
- Integrações: Stripe (pagamentos/webhooks), Cloudinary (media), Ably (chat/webhooks)

1.2 Frontend

- Next.js (App Router) + TypeScript
- Bun (runtime + package manager)
- Tailwind CSS
- Axios (camada de comunicação com a API via Gateway)
- Playwright (testes E2E/integração)

1.3 Infraestrutura

- Docker + Docker Compose (orquestração local/deploy)
- Postgres (container único com múltiplas bases de dados)
- RabbitMQ (broker + management)

1.4 Justificação das Escolhas

- **Java (Spring Boot) vs. Node.js:** Optámos por Java no backend devido ao nosso maior nível de conhecimento e controlo sobre a linguagem para gerir a lógica complexa dos microserviços.
- **Next.js vs. Express/React nativo:** Embora tenhamos experiência com Express, escolhemos o Next.js pela enorme facilidade de uso, convenções integradas de roteamento e otimização imediata.
- **TypeScript vs. JavaScript:** Escolhemos TypeScript em todo o frontend porque a tipagem estática previne erros em *runtime* e melhora a experiência de desenvolvimento e manutenção.
- **Tailwind CSS vs. CSS Tradicional:** O Tailwind foi selecionado porque a sua abordagem *utility-first* acelera drasticamente a criação da UI sem termos de sair dos ficheiros dos componentes.

2 Equipa, Gestão e Integração Contínua

2.1 Papéis da Equipa

A equipa dividiu as responsabilidades e o foco de desenvolvimento da seguinte forma:

- **Brandon Mejia:** *Fullstack*
- **Tiago Antunes:** *Frontend*
- **Luís Moreira (Daniel):** *Backend*
- **Miguel Correia:** *Backend*

2.2 Gestão de Projeto (SCRUM e Requisitos)

O projeto segue a metodologia ágil SCRUM. Optámos por não duplicar documentação neste relatório, uma vez que o levantamento de requisitos, a definição de atores, a gestão do *backlog* (com *user stories*) e o planeamento dos *sprints* estão totalmente centralizados e documentados no **Jira**.

Ao nível de código, utilizamos o Git com uma política de *Pull Requests*, onde o código é revisto antes de ser integrado na *branch* principal.

2.3 Integração Contínua (CI)

Implementámos *pipelines* de CI via GitHub Actions para garantir a estabilidade constante do código. Foram criados dois ficheiros de *workflow* distintos:

- **Backend CI:** Executa a construção e todos os testes unitários e de integração em Java.
- **Frontend CI:** Corre o *linter* e os testes E2E/API usando Playwright.

Estes *workflows* são disparados automaticamente a cada *commit* e *Pull Request*, prevenindo que código quebrado afete o ecossistema.

3 Objetivo

Este documento descreve como executar o projeto Nexus Estates com Docker (modo recomendado para deploy/demonstração), como executar em modo de desenvolvimento, qual a stack tecnológica usada, padrões de projeto aplicados e um resumo dos testes existentes.

Para mais detalhes operacionais, consulta:

- README.md (raiz)
- backend/README.md
- frontend/README.md

4 Como correr o projeto (Docker – Deploy/Demonstração)

4.1 Comando único (levanta tudo)

Na raiz do repositório:

```
docker compose -f infrastructure/docker-compose.deploy.yml up -d --build
```

4.2 Acessos e Conta de Teste

- Frontend: <http://localhost:3000>
- API Gateway: <http://localhost:8080>
- Swagger UI (via Gateway): <http://localhost:8080/swagger-ui.html>

Conta de Demonstração (com dados preenchidos):

- Email: dev@nexus.com
- Password: 12345

4.3 Parar e limpar

```
docker compose -f infrastructure/docker-compose.deploy.yml down
```

4.4 Como a comunicação funciona (rede interna)

O `docker-compose.deploy.yml` define uma rede privada (isolada) chamada `nexus-net`. Dentro desta rede:

- O Docker fornece **DNS interno**: o nome do serviço no Compose (ex.: `booking-service`) vira hostname.
- Os serviços comunicam por `http://<service-name>:<port>` sem depender de IPs fixos.
- Postgres e RabbitMQ ficam acessíveis internamente como `postgres` e `rabbitmq`.

Exposição de portas (segurança por defeito):

- Apenas **frontend (3000)** e **api-gateway (8080)** expõem `ports` para o host.
- Microserviços internos, Postgres e RabbitMQ não expõem portas para fora (continuam acessíveis dentro da `nexus-net`).

4.5 Configuração por variáveis de ambiente (.env por serviço)

Para simplificar o deploy, cada microserviço tem um ficheiro de variáveis dedicado em `infrastructure/env/` (um por serviço). Estes ficheiros:

- Definem `SPRING_DATASOURCE_*` apontando para `postgres`
- Definem `SPRING_RABBITMQ_*` apontando para `rabbitmq`
- Definem URLs internos (`*_SERVICE_URL`) apontando para `http://<service-name>:<port>`

Isto garante que os fallbacks `localhost` presentes em `application.properties` são substituídos automaticamente em Docker.

5 Como correr em DEV (infra em Docker + apps localmente)

5.1 Levantar apenas Postgres + RabbitMQ

```
docker compose -f infrastructure/docker-compose.yml up -d
```

5.2 Backend local (Spring Boot)

- Recomendado: abrir `backend/` no IntelliJ e iniciar serviços (começar pelo `api-gateway`).
- Alternativa terminal (na pasta `backend`):

```
mvn -pl api-gateway -am spring-boot:run
```

5.3 Frontend local (Next.js + Bun)

```
cd frontend
bun install
bun dev
```

6 Padrões de projeto e práticas (backend)

6.1 Separação por camadas (MVC + Service + Repository)

Os microserviços seguem uma organização típica:

- **controller**/: endpoints REST (camada HTTP)
- **service**/: regras de negócio e orquestração
- **repository**/: acesso a dados (Spring Data)
- **entity**/: modelos JPA
- **dto**/: contratos (requests/responses)
- **config**/: configuração Spring (security, rabbit, clients, etc.)
- **messaging**/: consumers/publishers e integração com RabbitMQ (quando aplicável)
- **exception**/: exceções e handlers
- **audit**/: contexto de actor + Envers (em serviços que auditam)

6.2 Proxy Pattern (HTTP declarativo entre microserviços)

O backend usa clientes HTTP declarativos do Spring (gerados em runtime), seguindo o **Proxy Pattern**:

- Interfaces em `client/NexusClients.java` definem endpoints.
- `config/WebClientConfig.java` constrói `RestClient` com base URL configurável e gera proxies via `HttpServiceProxyFactory`.

Isto reduz acoplamento e centraliza os contratos de comunicação.

6.3 Strategy Pattern (ex.: faturação no finance-service)

O `finance-service` encapsula o comportamento de emissão de faturas em estratégias:

- `InvoiceProviderStrategy` (interface)
- Implementações: `MockInvoiceProviderStrategy`, `MoloniInvoiceProviderStrategy`, `NoopInvoiceProvid`

Assim, o provider pode variar por configuração sem alterar a lógica central (`InvoiceOrchestrator`).

6.4 Resiliência (Circuit Breaker + Retry)

O `sync-service` aplica Resilience4j em chamadas externas (evita cascatas e dá degradação controlada):

- Circuit Breaker para reduzir falhas repetidas
- Retry com backoff para erros transitórios

6.5 Eventos assíncronos e consistência eventual

- RabbitMQ para publicação/consumo de eventos (ex.: booking created/status updated).
- Fluxos tipo Saga (coreografada) para consistência eventual entre serviços.

7 Testes (backend)

Os testes do backend estão por microserviço em `<module>/src/test/java`. Para ver a lista completa (com nomes de testes), consulta `<module>/TESTS.md`.

7.1 api-gateway

- **Filtros e segurança:** validações do JWT, rotas públicas vs protegidas e regras de segurança do gateway.
- **Principais classes:** `AuthenticationFilterTest`, `RouteValidatorTest`, `JwtUtilTest`.

7.2 booking-service

- **API e validações:** criar reservas, validar payload, conflitos de datas e leituras por utilizador/propriedade.
- **Domínio:** regras (min/max noites, lead time), overlaps e consistência de estado.
- **Eventos/RabbitMQ:** publicar e consumir eventos, incluindo bloqueios de calendário.
- **Pagamentos:** integração com o `finance-service` via client declarativo.
- **Principais classes:** `BookingControllerTest`, `BookingServiceTest`, `BookingRepositoryTest`, `BookingEventPublisherTest`, `CalendarBlockConsumerTest`, `BookingPaymentServiceTest`.

7.3 property-service

- **CRUD e endpoints:** propriedades e amenities (criar/listar/editar/apagar), endpoints expandidos e histórico.
- **Regras e preços:** regras base, sazonalidade e cotações (quote).
- **Permissões:** ACL por níveis (`PRIMARY_OWNER/MANAGER/STAFF`) e validações.
- **Upload:** parâmetros de upload e integração com Clouinary.
- **Auditoria:** `ActorContext` e Envers.

7.4 sync-service

- **Config e filas:** bindings RabbitMQ + DLQ e testes de configuração.
- **Integração externa:** cliente HTTP, autenticação externa e resiliência (inclui testes de integração).
- **Ferramentas admin:** endpoints para DLQ e parsing/apply de ICS.
- **Webhooks e chat:** validações de assinatura, dispatch de webhooks e persistência/notificações.

7.5 finance-service

- **Pagamentos:** criação/confirmação e persistência de pagamentos.
- **Webhooks Stripe:** validação de assinatura e idempotência.
- **Faturação:** orquestração de invoice e estratégias (mock/moloni/noop).

7.6 user-service

- **Auth:** registo/login e JWT.
- **Segurança:** filtro JWT, roles e headers do gateway.
- **Recuperação de password:** controllers e serviço (tokens, expiração, privacidade).
- **Integrações:** endpoints para integrações externas.
- **Auditoria:** `ActorContext` e Envers.

7.7 Como correr os testes (backend)

Na pasta `backend`:


```
mvn test
```

Ou por módulo:

```
mvn -pl booking-service -am test
```

8 Testes (frontend) — integração/E2E (Playwright)

Os testes do frontend estão em `frontend/e2e/api`. Eles testam os fluxos principais via API Gateway (por defeito `http://localhost:8080/api`).

- `auth.spec.ts`: registo/login, permissões por role, perfil, integrações e webhooks (inclui casos sem token e casos de segurança como IDOR).
- `properties.spec.ts`: CRUD de propriedades, validações, regras/sazonalidade, permissões (ACL) e endpoints de upload.
- `bookings.spec.ts`: criar reservas, validar conflitos de datas/overlaps, bloqueios e listagens do utilizador.
- `amenities.spec.ts`: CRUD de amenities.

8.1 Como correr testes do frontend

Pré-condição: o backend deve estar acessível (ex.: via Docker deploy, gateway em `http://localhost:8080`).

Na pasta `frontend`:

```
bun test:e2e
```

Modo UI:

```
bun test:e2e:ui
```

References

- [1] Google Cloud. *O que é a arquitetura de microsserviços?*.
Disponível em: <https://cloud.google.com/learn/what-is-microservices-architecture?hl=pt-BR>
- [2] Docker. *Docker Documentation*.
Disponível em: <https://docs.docker.com/>
- [3] Vercel. *Next.js - The React Framework for the Web*.
Disponível em: <https://nextjs.org/>
- [4] Bun. *Fast all-in-one JavaScript runtime*.
Disponível em: <https://bun.com/>
- [5] Tailwind Labs. *Tailwind CSS*.
Disponível em: <https://tailwindcss.com/>
- [6] Microsoft. *TypeScript*.
Disponível em: <https://www.typescriptlang.org/>
- [7] VMware. *Spring Boot*.
Disponível em: <https://spring.io/projects/spring-boot>
- [8] TPointTech. *Java Tutorial*.
Disponível em: <https://www.tpointtech.com/java-tutorial>
- [9] PostgreSQL Global Development Group. *PostgreSQL Documentation*.
Disponível em: <https://www.postgresql.org/docs/>
- [10] YouTube (Amigoscode). *Spring Boot Tutorial*.
Disponível em: <https://www.youtube.com/watch?v=gJrjgg1KVL4>
- [11] YouTube (Bouali Ali). *Spring Boot Tutorial - Full Course*.
Disponível em: https://www.youtube.com/watch?v=YY_hf0F0IcU